**Assignment 2. Topic: Hand Gesture Recognition**

**A1 baselines vs SOTA MobileNetV4, with a web demo**

**Aiya Zhakupova SE-2409**

**GitHub: Aiya594/aitu-cv-asik2**

**Video: https://youtu.be/PIESAwjPEc4**

## INTRODUCTION

Hand gesture recognition is a computer vision task that aims to automatically identify the meaning of a hand gesture from an image. It is a useful application of image classification because it can be used in areas such as human-computer interaction, accessibility systems, education, and touchless interfaces. However, recognizing hand gestures from photographs can be challenging because the appearance of a hand may change depending on its position, orientation, lighting conditions, background, and distance from the camera.

In Assignment 1 (A1), two models were trained from scratch to recognise three gestures (one, two and three raised fingers): Logistic Regression on raw pixels and a small convolutional neural network (CNN). Re-evaluated on the expanded dataset and a separate test set, both reach only about 40% accuracy, barely above the 33% chance level. Assignment 2 (A2) keeps the same topic and data and adds a state-of-the-art (SOTA) component: MobileNetV4 [5], a 2024 Google architecture pretrained on ImageNet [2] and fine-tuned on our gestures. All three models are compared on the same held-out test set. The study adds a robustness test under simulated conditions, an ablation, Grad-CAM explanations [7], and a web application that runs the best model.

## DATASET UPDATE

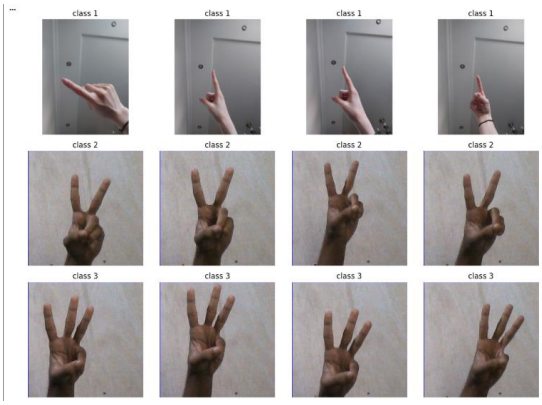
The dataset combines images from a publicly available Kaggle dataset [4] with photographs collected by me. The Kaggle images serve as additional samples for the three gesture classes, while the collected photographs provide original examples of the same gestures. The images are organised in a standard "one folder per class" layout, where the folder name serves as the class label.

For A2 the dataset was expanded from 300 to 2,700 images: 800 per class in the train folder and 100 per class in a separate validation folder. Exact duplicates (files saved twice, e.g. 1000.jpg and 1000(1).jpg) were detected with MD5 hashes and removed: 43 images, all in class 1. This guarantees that no picture appears in both training and test data. The validation folder is used only as the held-out test set, and a stratified 15% of the train folder serves as the validation split for model selection (Table 1).

| Class | Meaning | Train | Validation | Test | Total |
|---|---|---|---|---|---|
| 1 | Hand showing one finger | 647 | 115 | 95 | 857 |
| 2 | Hand showing two fingers | 680 | 120 | 100 | 900 |
| 3 | Hand showing three fingers | 680 | 120 | 100 | 900 |
| **Total** | | **2007** | **355** | **295** | **2657** |

Table 1. Dataset composition after removing duplicates

**Synthetic share.** No generated images were added to the training data. Synthetic images are used only for evaluation: each of the 295 test photos is transformed programmatically into six conditions (low light, over-exposure, blur, noise, rotation, new background), giving 1,770 synthetic test images. They are labelled as synthetic in every table and figure. Heavy text-to-video generators such as Step-Video-T2V were not used; transformations of real photos are cheap, fully reproducible and keep the true label.



Picture 1. Example images from the dataset

## PREPROCESSING AND AUGMENTATION

**Resizing.** A neural network needs a fixed input shape, so all images are resized: to 128×128×3 for the two A1 models (unchanged from A1) and to 224×224×3 for MobileNetV4, the resolution it was pretrained at.

**Normalization.** Pixel values 0-255 are divided by 255 so that inputs lie in [0, 1], which gives more stable gradients. For Logistic Regression each image is flattened into 128·128·3 = 49,152 numbers. MobileNetV4 additionally subtracts the ImageNet channel mean and divides by its standard deviation, exactly as during pretraining; otherwise the pretrained filters would receive inputs with a different distribution.

**Augmentation.** Although the training set is now about ten times larger, all photos come from very similar sessions, so a flexible model can still memorise them. Training images are therefore transformed randomly in every epoch with label-preserving changes. The CNN uses the A1 pipeline: horizontal flip, rotation up to +-29°, zoom and contrast change up to +-15%, and translation up to +-10%. MobileNetV4 uses the equivalent torchvision pipeline: random resized crop (75-100% of the area), horizontal flip, rotation up to +-15° and colour jitter (brightness, contrast and saturation +-30%, hue +-5%). Validation and test images are never augmented.

## MODELS

### Baseline model: Logistic Regression

The baseline is multinomial Logistic Regression trained on the flattened, normalised pixel vectors (scikit-learn [3], max_iter = 2000). It learns a linear decision boundary in a 49,152-dimensional space and has no notion of spatial structure: a finger that shifts a few pixels looks like a completely different input. It is a deliberately weak lower bound that any more sophisticated model must beat.

### A1 model: a small CNN trained from scratch

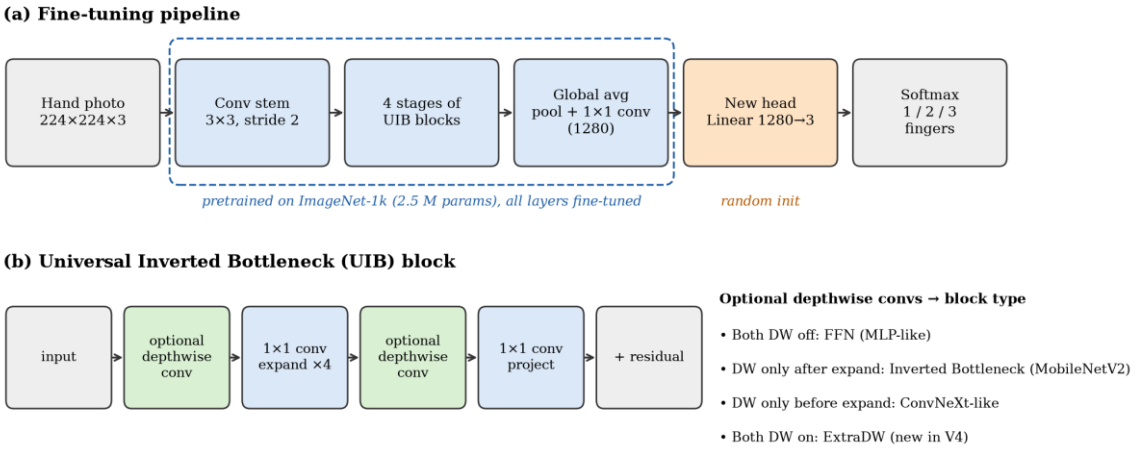
The CNN applies small learnable 3×3 filters at every position of the image, so a pattern such as a finger outline is recognised wherever it appears; max-pooling halves the resolution and adds tolerance to small shifts. The network has three Conv-ReLU-MaxPool blocks with 16, 32 and 64 filters, a 64-unit dense layer, dropout 0.5 and a softmax output: 1,072,419 parameters, 98% of them in the single dense layer. It is trained with Adam, batch size 10, up to 25 epochs with early stopping on the validation loss (patience 6). Both A1 models are retrained in A2

without changes, only on the larger dataset, so the comparison isolates the effect of the SOTA model.

**SOTA model: MobileNetV4 (Qin et al., 2024)**

MobileNetV4 [5] is the fourth generation of Google's MobileNet family, presented at ECCV 2024. Its goal is universal efficiency: the best accuracy-to-latency trade-off on mobile CPUs, GPUs, DSPs and dedicated accelerators at the same time, instead of being tuned for a single device.

Its key idea is the Universal Inverted Bottleneck (UIB) block (Picture 2b). It extends the inverted bottleneck of MobileNetV2 with two optional depthwise convolutions: one before the 1×1 expansion and one between expansion and projection. Switching them on or off turns one block into four known designs: the inverted bottleneck, a ConvNeXt-like block, a feed-forward (FFN) block, or the new "ExtraDW" block with both. A neural architecture search then chooses the variant, kernel size and width of every layer, so the network spends computation on spatial mixing only where it helps. Larger hybrid variants also add a Mobile MQA attention block; the conv-small variant used here is fully convolutional.




Picture 2. MobileNetV4: (a) the fine-tuning pipeline used in this work, (b) the UIB block (after [5])

We use mobilenetv4_conv_small from the timm library [6], pretrained on ImageNet-1k. Its 1000-class classifier is replaced with a new 3-class linear layer and the whole network is fine-tuned (Picture 2a) with AdamW (learning rate $5 \cdot 10^{-4}$, weight decay 0.05), cosine learning-rate decay, label smoothing 0.1 and batch size 32 for 15 epochs; the epoch with the best validation accuracy is kept. Training takes under 5 minutes on a Colab T4 GPU.

**Why it helps this topic.** The A1 experiments showed that the main difficulty is data, not model capacity: about 2,000 photos of one hand on one background are not enough to learn edges, skin and finger shapes from random weights. A pretrained network already contains these general visual features from 1.2 million ImageNet images, so fine-tuning only has to learn how to count fingers. MobileNetV4's efficiency also makes real-time inference on an ordinary laptop CPU possible, which a webcam demo needs.

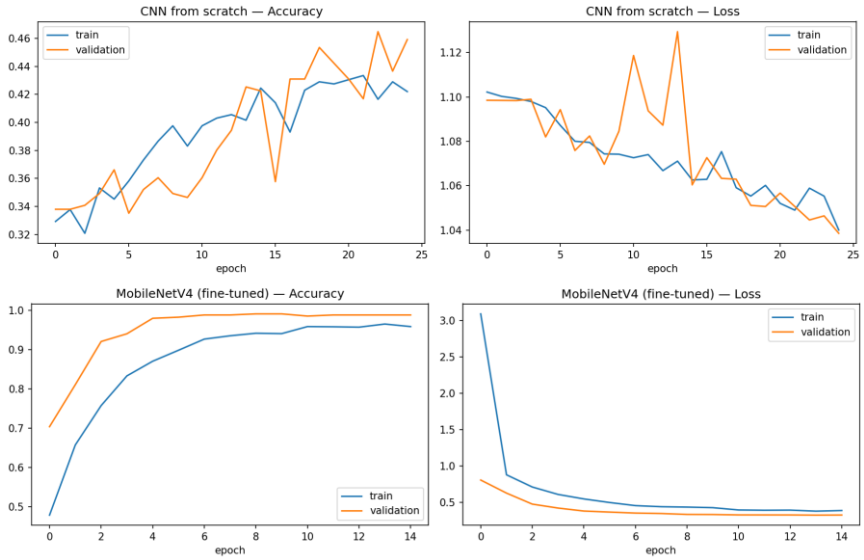
## EXPERIMENTS

All models are evaluated on the same 295 held-out test images. The metrics are accuracy and macro-averaged precision, recall and F1, so each class counts equally. Latency is measured on a CPU, one image at a time, averaged over 100 runs after a warm-up (Colab CPU, 1 thread), without preprocessing.

| **Model** | **Acc.** | **Prec.** | **Recall** | **F1** | **CPU, ms** | **Params, M** | **Train, s** |
|---|---|---|---|---|---|---|---|
| Logistic Regression (A1) | 0.373 | 0.360 | 0.372 | 0.354 | 0.3 | 0.15 | 491 |
| CNN from scratch (A1) | 0.420 | 0.442 | 0.419 | 0.405 | 20.8 | 1.07 | 60 |
| **MobileNetV4 (SOTA)** | **0.986** | **0.987** | **0.986** | **0.986** | **14.4** | **2.50** | **285** |

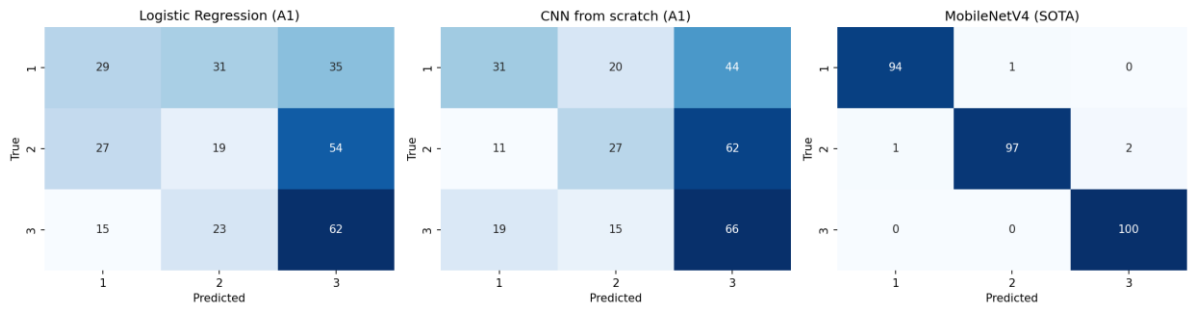
Table 2. Test-set results (295 images)

MobileNetV4 more than doubles the accuracy of both baselines and makes only 4 errors on 295 images. The A1 models are only slightly above chance: both over-predict class 3 and recognise class 2 worst (recall 0.19 and 0.27, Picture 4). Although it has more parameters, MobileNetV4 runs faster than the CNN on the CPU (14.4 vs 20.8 ms), which is about 70 images per second.




Picture 3. Training curves: CNN from scratch (top) and fine-tuned MobileNetV4 (bottom)

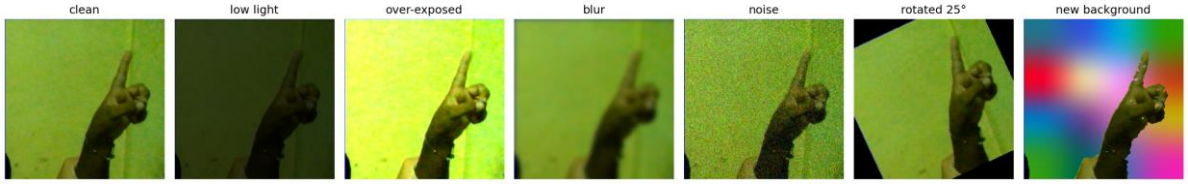
The CNN learns very slowly: after 25 epochs its training accuracy is only 0.42 and the loss has fallen from 1.10 (the chance level) to 1.04, so it underfits rather than overfits. MobileNetV4 reaches 70% validation accuracy after the first epoch, 92% after three and 99.2% after nine; both curves then flatten, so training has converged. Validation accuracy is above training accuracy because the training images are augmented and therefore harder.




Picture 4. Confusion matrices on the test set (rows: true class, columns: predicted class)

### Robustness under simulated conditions

All training photos share one green wall and indoor light, while a webcam will not. Each test image was therefore transformed into six synthetic conditions (Picture 5): brightness ×0.35 (low light), ×1.8 (over-exposure), Gaussian blur (radius 3), Gaussian noise ($\sigma = 25$), a 25° rotation, and a new background (the green wall replaced by random colours with a simple chroma key).




Picture 5. The six synthetic test conditions applied to one test photo (synthetic, evaluation only)

| Condition | Logistic Regression | CNN | MobileNetV4 |
|---|---|---|---|
| Clean (real photos) | 0.373 | 0.420 | 0.986 |
| Low light (synthetic) | 0.393 | 0.305 | 0.847 |
| Over-exposed (synthetic) | 0.386 | 0.363 | 0.895 |
| Blur (synthetic) | 0.420 | 0.376 | 0.851 |
| Noise (synthetic) | 0.386 | 0.403 | 0.841 |
| Rotated 25° (synthetic) | 0.329 | 0.346 | 0.973 |
| New background (synthetic) | 0.288 | 0.383 | 0.902 |

Table 3. Test accuracy under each condition

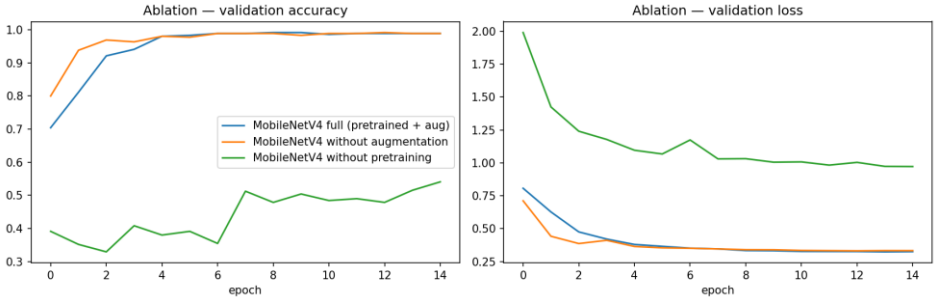
MobileNetV4 stays above 84% in every condition. It handles rotation (97.3%) and a new background (90.2%) best; noise, low light and blur (84-85%) are its weak spots. The baselines stay at 29-42% everywhere.

### Ablation

Two components of the MobileNetV4 pipeline were removed one at a time, and the model was retrained with otherwise identical settings (Table 4, Picture 6).

| Variant | Test accuracy | Macro-F1 | Mean acc. over 7 conditions |
|---|---|---|---|
| **Full (pretrained + augmentation)** | **0.986** | **0.986** | **0.899** |
| Without augmentation | 0.976 | 0.976 | 0.762 |
| Without pretraining (random init) | 0.549 | 0.541 | 0.440 |

Table 4. Ablation of MobileNetV4

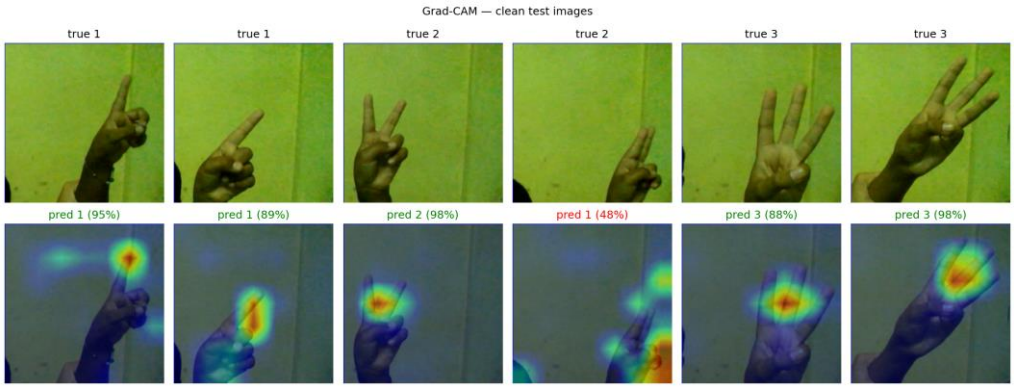



Picture 6. Validation accuracy and loss of the three ablation variants

Pretraining is decisive: the same architecture trained from random weights reaches only 54.9%, similar to the A1 CNN, and its validation loss stays near 1.0. Augmentation hardly changes clean accuracy (97.6% vs 98.6%) but strongly affects robustness: the mean accuracy over the seven conditions drops from 0.899 to 0.762 without it.

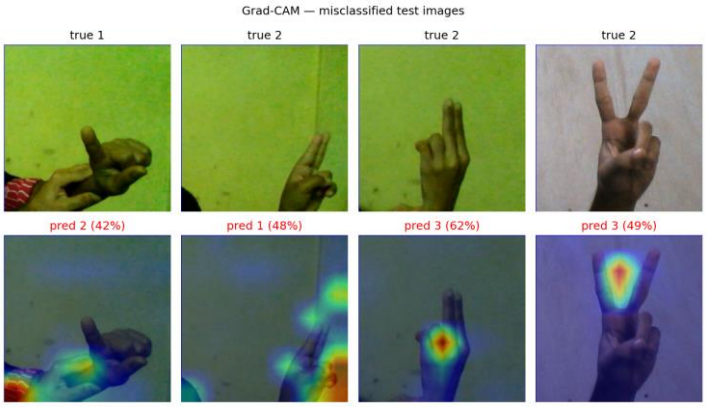
## EXPLAINABILITY

Grad-CAM [7] weights the feature maps of the last convolutional stage of MobileNetV4 by the gradient of the predicted class score and overlays the result as a heatmap: red marks the regions that increased the score. On correctly classified images the heat lies on the raised fingers rather than on the background (Picture 7), so the model uses the right evidence.




Picture 7. Grad-CAM on test images (green title: correct, red title: wrong)

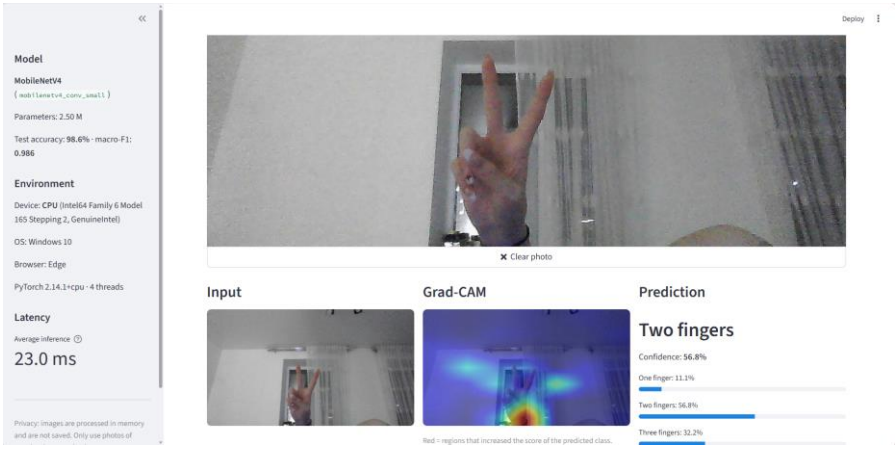
All four test errors (Picture 8) are unusual images: a second hand in the frame, a hand cut off at the image border, two fingers pressed together with a half-raised thumb, and the only photo with a grey instead of a green wall. The model's confidence was low on each of them (42-62%), so low confidence is a useful warning signal; the web app shows a warning below 60%. With the synthetic new background the heat sometimes spreads onto the coloured background, which explains the lower accuracy under that condition.




Picture 8. Grad-CAM on all four misclassified test images

## WEB APPLICATION

The fine-tuned MobileNetV4 model is deployed as a Streamlit [8] web application (app/app.py). The user uploads a photo or takes one with the webcam; the app resizes and normalises it exactly as in training, runs MobileNetV4 on the CPU and shows the predicted class, the probability of each class, a Grad-CAM overlay and the inference time (Picture 9). The sidebar reports the model's test accuracy, the environment (CPU, operating system, browser, PyTorch threads) and the average latency over all predictions, about 14 ms on a laptop CPU. The app is built only for MobileNetV4: at start-up it loads the checkpoint app/mobilenetv4_gesture.pt, which stores the weights, class names, input size, normalisation values and test metrics, and it does not compare or choose between models. It therefore works with any MobileNetV4 checkpoint placed in the app/ folder, whether that run is the best or a weaker one; the Logistic Regression and CNN baselines are not used in the app.




Picture 9. The web application classifying a two-finger photo

## ETHICS AND LIMITATIONS

**Consent and privacy.** The collected photos show only my own hand and were taken with my consent; the Kaggle images [4] are publicly released. The app stores nothing and states this in its interface.

**Bias.** The data contain one person (one skin tone and hand size), mostly one green background and indoor lighting. The robustness test and Grad-CAM show that accuracy drops with other

backgrounds and lighting, and it is likely lower for other skin tones and hands, which were not tested. A more diverse dataset is the main improvement needed before any real use.

**Synthetic media and disclosure.** All condition images are generated from real test photos and labelled as synthetic; no synthetic image was used for training, and no generated faces or people appear in this work.

**Misuse risks.** Finger counting itself is low-risk, but camera-based recognition could be misused for covert monitoring. The app uses the camera only after the browser asks for permission and never records.

## CONCLUSION

Assignment 2 upgraded the A1 hand-gesture classifier with a SOTA pretrained model. On the same 295-image test set, fine-tuned MobileNetV4 reached 98.6% accuracy and macro-F1 0.986, compared with 37.3% for Logistic Regression and 42.0% for the CNN trained from scratch, while running at about 14 ms per image on a CPU. The ablation shows that ImageNet pretraining is the main reason for the improvement and that augmentation keeps the model robust to lighting, blur and new backgrounds. Grad-CAM confirms that the model looks at the fingers, and its few errors are unusual images on which it is also uncertain. The main limitation is the narrow dataset (one person, one background), so future work should add more people, backgrounds and lighting conditions.

## REFERENCES


[1] TensorFlow. TensorFlow API Documentation. Available at: https://www.tensorflow.org/api_docs

[2] Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., and Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. CVPR 2009.

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12.

[4] A. Dalvi, "Hand Gestures Dataset," Kaggle, 2020. Available: https://www.kaggle.com/datasets/aadeshdalvi/hand-gestures-dataset

[5] Qin, D. et al. (2024). MobileNetV4: Universal Models for the Mobile Ecosystem. ECCV 2024. arXiv:2404.10518. Available at: https://arxiv.org/abs/2404.10518

[6] Wightman, R. PyTorch Image Models (timm). Available at: https://github.com/huggingface/pytorch-image-models

[7] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017.

[8] Streamlit. Streamlit Documentation. Available at: https://docs.streamlit.io

[9] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.